Python-Based Network Scanner & Nmap Automation Tool

InternNova Cybersecurity Internship · Week 6 · Offensive Security – Project 1 | Kanagala Mahesh · B.Tech CSE, DRK Institute of Science and Technology · CEH

1. Why I chose this project

I am training as an ethical hacker, and reconnaissance is the first step of every security assessment. Automating Nmap with Python let me practise scripting while understanding how scans really work. It also fits my home lab (Kali Linux + Metasploitable2), so I could test it safely on systems I own.

2. Technologies / Tools Used

Python 3 (standard library only: `subprocess`, `xml.etree.ElementTree`, `ipaddress`, `csv`, `json`), Nmap, Kali Linux, VirtualBox (host-only network), Metasploitable2 as the target, Git/GitHub. All free tools.

3. How I completed the project

I built a menu-driven CLI with five scan profiles: host discovery (`-sn`), quick top-100 scan (`-F`), full TCP connect scan (`-sT -p-`), service/version detection (`-sV -F`) and full-port version detection (`-sV -p- -T4`). The tool runs Nmap through `subprocess` with an argument list (no shell) and XML output (`-oX -`), parses it into rows (host, port, protocol, state, service, version), prints a formatted table, and exports timestamped TXT, JSON and CSV reports. For responsible scanning, it flags non-private targets, requires the user to confirm ownership or written permission before every scan, and uses per-scan timeouts and error handling.

4. What I learned

- How TCP connect scans differ from version detection in speed, coverage and detail
- Reading open ports and service banners to map a target's attack surface
- Parsing Nmap XML and exporting structured data (JSON/CSV/TXT) in Python
- Safe use of `subprocess` and the legal and ethical rules of scanning

5. Challenges Faced

- **Empty VERSION column:** my first full scan listed ports but no versions because it did not use `-sV`. I added a full-port version-detection profile, which identified all 30 services.
- **Scan time:** version detection on all 65,535 ports is slow and exceeded my fixed 15-minute limit, so I gave each profile its own timeout.
- **Unreliable output:** parsing Nmap's text output is fragile, so I switched to XML parsing, which also handles hosts with no open ports.

6. Final Outcome

A working scanner, tested only on my own lab, that reports every open port with its service and version. The full scan of Metasploitable2 (192.168.56.102) found **30 open TCP ports**. Highlights:

Port	Service	Detected version	Why it matters
21	ftp	vsftpd 2.3.4	Release with a known backdoor (CVE-2011-2523)
23 / 512–514	telnet, r-services	Linux telnetd, netkit rexecd/rshd	Unencrypted remote access, weak authentication
1524	bindshell	Metasploitable root shell	Open root shell, no authentication
3632	distccd	distccd v1 (GNU 4.2.4)	Known remote command execution issue (CVE-2004-2687)
3306 / 5432	mysql, postgresql	MySQL 5.0.51a, PostgreSQL 8.3.x	Databases exposed to the network, very outdated
80 / 8180	http	Apache 2.2.8, Tomcat/Coyote 1.1	Outdated web stack, extra Tomcat admin surface

Results were saved in all three required formats (TXT, JSON, CSV). Code, sample output and screenshots are on GitHub: github.com/Arjun-7x/nmap-automation-tool